Matteo Palmonari
matteo.palmonari@unimib.it

INSID&S Lab
***In**teraction and **S**emantics for*
***I**nnovation with **D**ata **&** **S**ervices*

2.1- Part II Knowledge Graphs & RDF

Knowledge Graphs & RDF: Outline

- Knowledge Graphs
  - What they are, intuitively
  - Why they are useful
  - What they are, precisely
- The RDF data model
  - RDF

Part I

Part II – This presentation

Knowledge Graphs & RDF: Outline

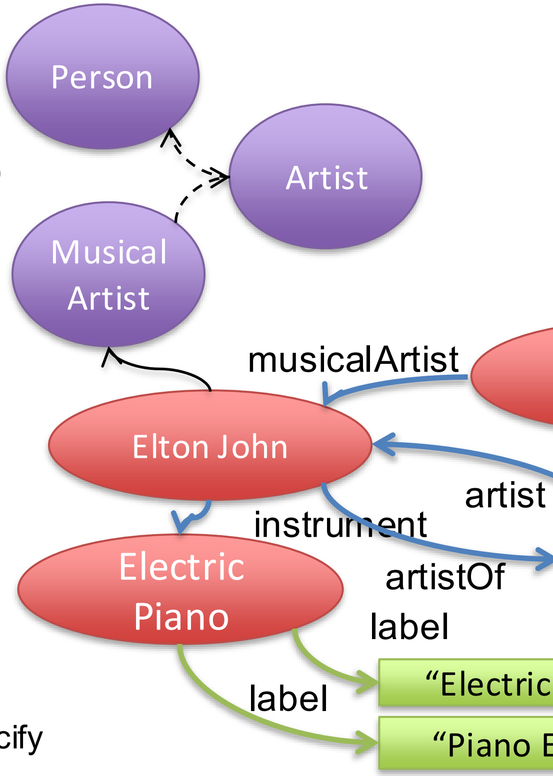
- Knowledge Graphs
  - What they are, intuitively
  - Why they are useful
  - **What they are, precisely**
- The RDF data model
  - RDF

Part I

Part II – This presentation

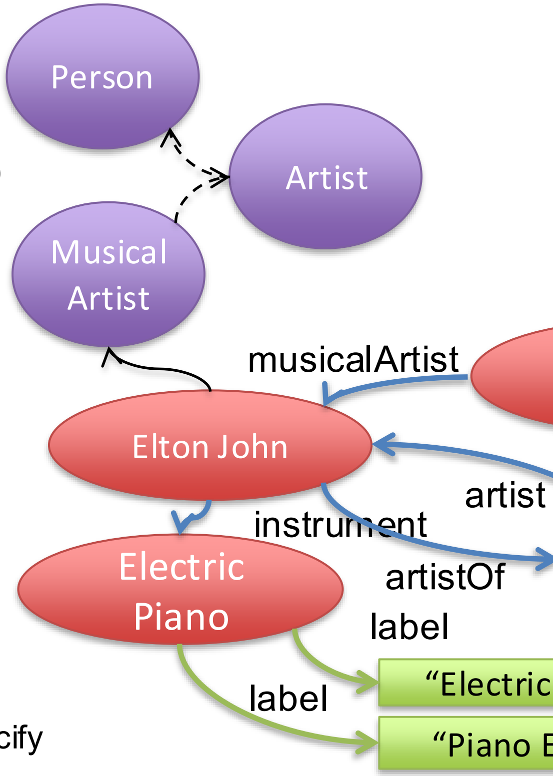
Minimal Knowledge Graph Definition

- A quintuple: <*E*,*L*,*T*,*P*,***A***>
  - *E*: a set of entity id symbols (= constants)
  - *L*: a set of literals, i.e., strings, numbers, etc. (= constants)
  - *T*: a set of type symbols (= unary predicate symbols)
  - *P*: a set of relation symbols (= binary predicate symbols)
  - ***A***: a set of axioms in a language ….
- *A*: a set of **axioms** in a language (≈ subset of FOL)
  - A-facts:
    - Types of entities and literals: $\boldsymbol{t(e)}$ | $\boldsymbol{t(l)}$ - with $e \in E$ and $l \in L$
    - Relations between entities: $\boldsymbol{r(e_1, e_2)}$ | $\boldsymbol{r(e, l)}$ – with $e_i \in E$ and $l \in L$
  - A-general-axioms:
    - Depends on the language….
    - Minimal requirements: axioms with the form $\forall \boldsymbol{x}$ ($\boldsymbol{t_1(x)} \rightarrow \boldsymbol{t_2(x)}$) that specify subtype dependencies that define a partial order over *T*



Minimal Knowledge Graph Definition

- A quintuple: $\langle E,L,T,P,\boldsymbol{A}\rangle$
  - E: a set of entity id symbols (= constants)
  - L: a set of literals, i.e., strings, numbers, etc. (= constants)
  - T: a set of type symbols (= unary predicate symbols)
  - P: a set of relation symbols (= binary predicate symbols)
  - $\boldsymbol{A}$: a set of axioms in a language ….
- A: a set of **axioms** in a language (≈ subset of FOL)
  - A-facts:
    - Types of entities and literals: $\boldsymbol{t(e)}$ | $\boldsymbol{t(l)}$ - with $e \in E$ and $l \in L$
    - Relations between entities: $\boldsymbol{r(e_1, e_2)}$ | $\boldsymbol{r(e, l)}$ – with $e_i \in E$ and $l \in L$
  - A-general-axioms:
    - Depends on the language….
    - Minimal requirements: axioms with the form $\forall \boldsymbol{x}$ ($\boldsymbol{t_1(x)} \rightarrow \boldsymbol{t_2(x)}$) that specify subtype dependencies that define a partial order over T



Different Approaches to Represent, Store & Query Knowledge Graphs

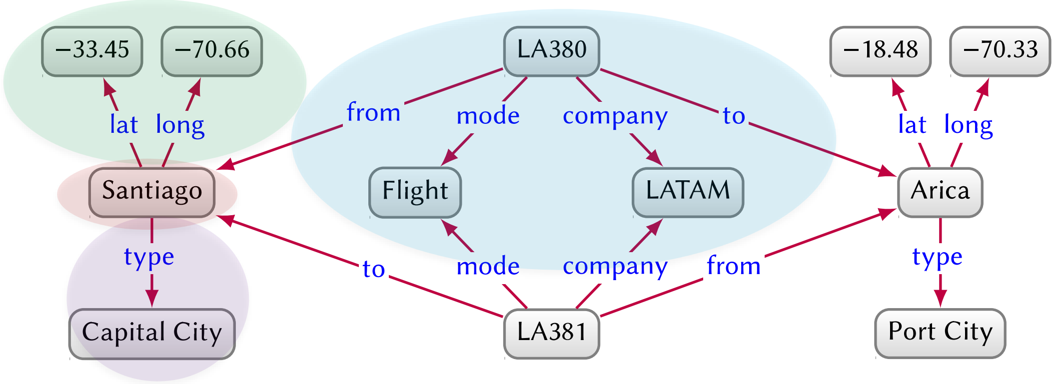


Fig. 3. Directed edge-labelled graph with companies offering flights between Santiago and Arica

Directed edge-labeled graph model: nodes / edges (that's it!)

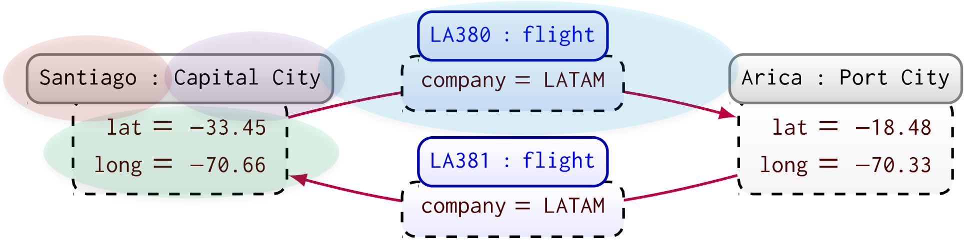


Fig. 4. Property graph with companies offering flights between Santiago and Arica

Property graph model: nodes, types, relationships, properties

…

Source: Hogan & al. Knowledge Graphs. https://arxiv.org/abs/2003.02320

Different Approaches to Represent, Store & Query Knowledge Graphs

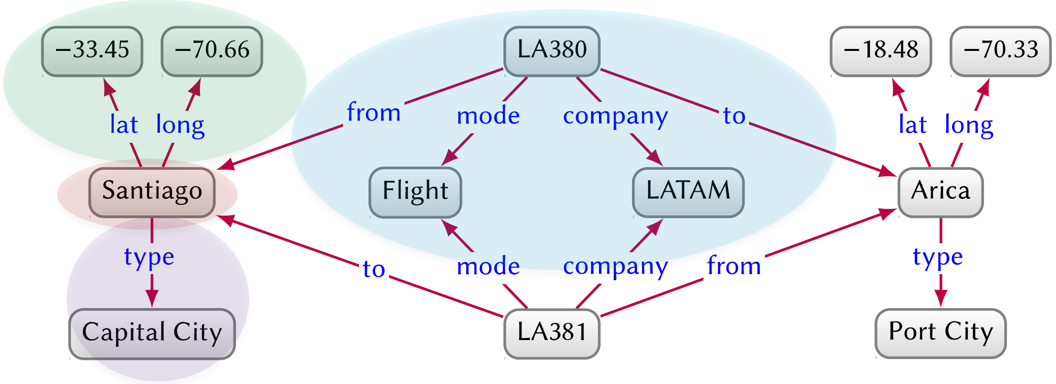


Fig. 3. Directed edge-labelled graph with companies offering flights between Santiago and Arica

Directed edge-labeled graph model: nodes / edges (that's it!)

Representation (**RDF**) + query (SPARQL) language:

All triple stores, incl. Virtuoso, AllegroGraph, Amazon Neptune

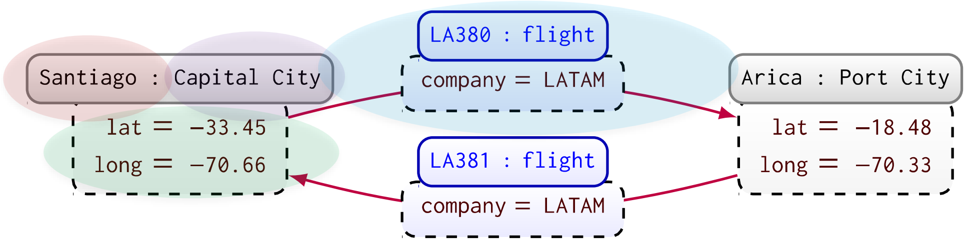


Fig. 4. Property graph with companies offering flights between Santiago and Arica

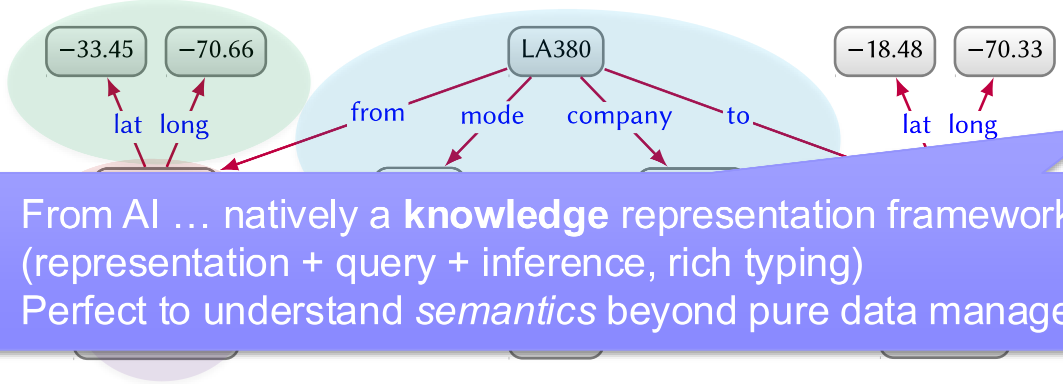
Property graph model: nodes, types, relationships, properties

Different languages in different databases:

native graph dbs, incl. Neo4J (Cypher), Amazon Neptune (Gremlin), and multimodal dbs, incl. OrientDB (Gremilin), ArangoDB (AQL)

Source: Hogan & al. Knowledge Graphs. https://arxiv.org/abs/2003.02320

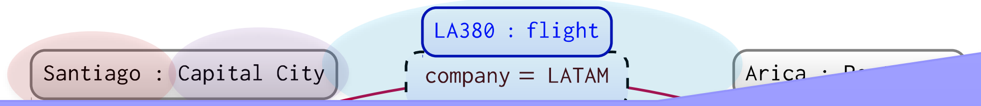
Different Approaches to Represent, Store & Query Knowledge Graphs



Directed edge-labeled graph model: nodes / edges (that's it!)

- From AI … natively a **knowledge** representation framework (representation + query + inference, rich typing)
- Perfect to understand *semantics* beyond pure data management

Fig. 3. Directed edge-labelled graph with companies offering flights between Santiago and Arica



Property graph model: nodes, types, relationships, properties

- From data management … natively a **data** representation framework (representation + query + graph analytics)
- Advanced solutions, e.g., inference | rich typing, can be hard-coded after a good understanding with RDF

Source: Hogan & al. Knowledge Graphs. https://arxiv.org/abs/2003.02320

Additional Resources - II

- Knowledge Graphs (useful in particular after next lessons)
  - Reference book (NEW!): A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, J. E. Labra Gayo, S. Kirrane, S. Neumaier, A. Polleres, R. Navigli, A. N. Ngomo, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, A. Zimmermann. **Knowledge Graphs.** Download at: https://arxiv.org/abs/2003.02320
  - Publicly available Knowledge Graphs: N. Heist, S. Hertling, D. Ringler, H. Paulheim. **Knowledge Graph on the Web.** Download at: https://arxiv.org/abs/2003.00719
  - (Some) Knowledge Graphs in industry: N. Noy, Y. Gao, A. Jain, A. Narayanan, A. Patterson, J. Taylor. **Industry-scale Knowledge Graphs: Lessons and Challenges.** Communications of the ACM, vol. 62 (8) (2019), pp. 36-43. Download at: https://research.google/pubs/pub48449/

Knowledge Graphs & RDF: Outline

- Knowledge Graphs
  - What they are, intuitively
  - Why they are useful
  - What they are, precisely
- **Knowledge Graphs & RDF**
  - RDF
  - Vocabularies for RDF

RDF:

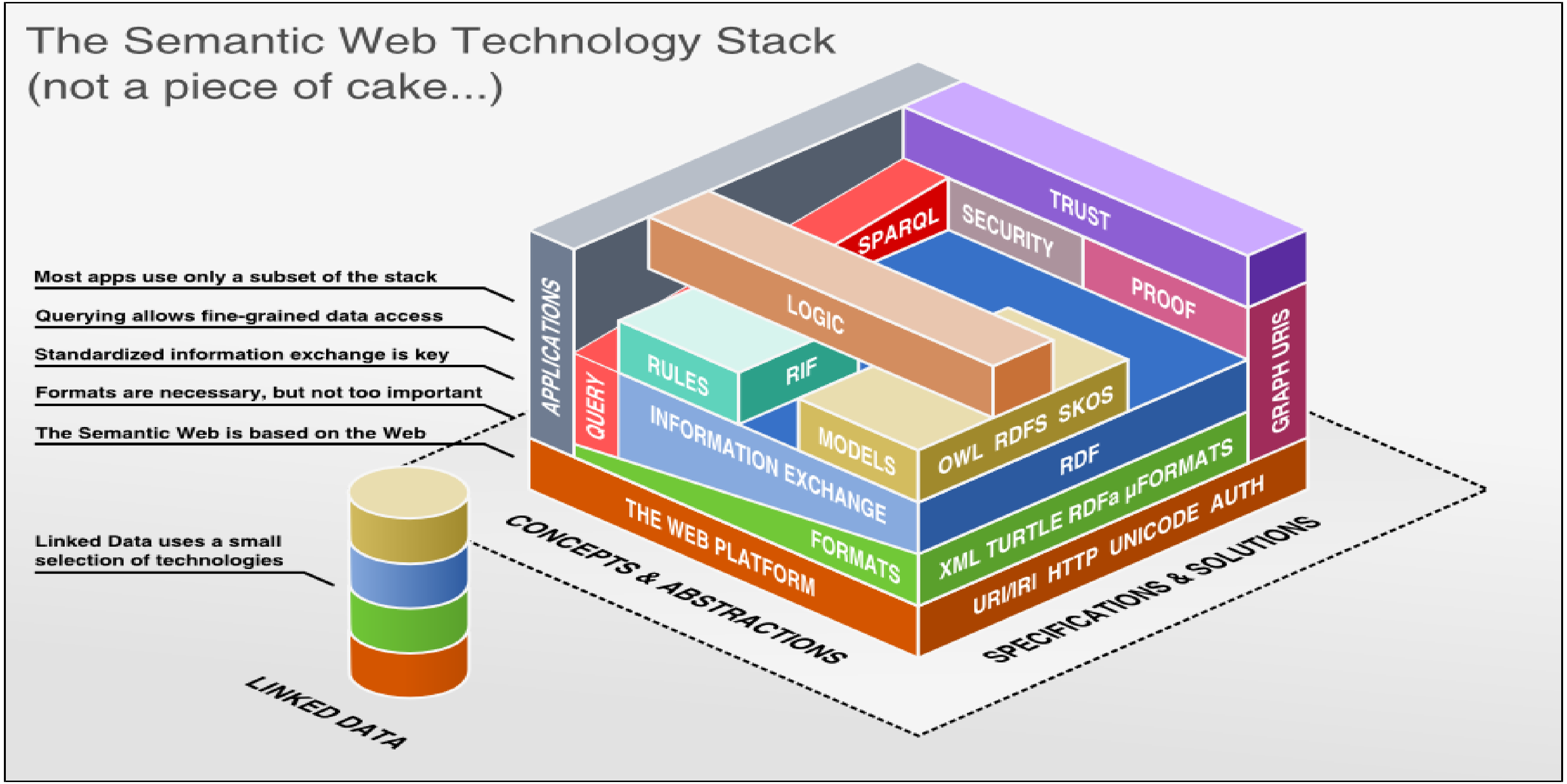
Resource Description Framework

RDF 1.1
A W3C Recommendation

W3C Recommendations

- W3C: World Wide Web Consortium
- Created to lead the Web to its full potential by developing common protocols that promote its evolution and ensure its interoperability.
- International industry consortium jointly run by the MIT Computer Science and Artificial Intelligence Laboratory (MIT CSAIL) in the USA, the European Research Consortium for Informatics and Mathematics (ERCIM) headquartered in France and Keio University in Japan. 350+ organizations are Members of W3C.
- Services provided by the Consortium include: a repository of information about the World Wide Web for developers and users, and various prototype and sample applications to demonstrate use of new technology.
- For more information see http://www.w3.org/

Semantic Web Stack Revised



http://bnode.org/blog/2009/07/08/the-semantic-web-not-a-piece-of-cake

RDF: Basic Principles

- RDF is a data model for representing data on the Web

  based on:
  - Triples – basic unit to organize the information
  - Directed (<u>labeled</u>) graphs – sets of triples
  - URIs – unique resource identifiers

RDF: Basic Principles

- RDF is a data model for representing data on the Web

  based on:

  - **Triples – basic unit to organize the information**
  - Directed (<u>labeled</u>) graphs – sets of triples
  - URIs – unique resource identifiers

RDF Triples // Statements

<Electric Piano, label, “Electric Piano”@en>
<Electric Piano, label, “Piano Elettrico”@it>
<Elton John, instrument, Electric Piano>
<Empty Sky, artist, Elton John>
<Sails, musicalArtist, Elton John>
<Sails, album, Empty Sky>
<Elton John, instrument, Electric Piano>
<Elton John, type, Musical Artist>
<Sails, type, Single>
<Empty Sky, type, Album>
<Musical Artist, subClassOf, Artist>
<Artist, subClassOf, Person>
<Album, subClassOf, Musical Work>
<Single, subClassOf, Musical Work>

RDF: Basic Principles

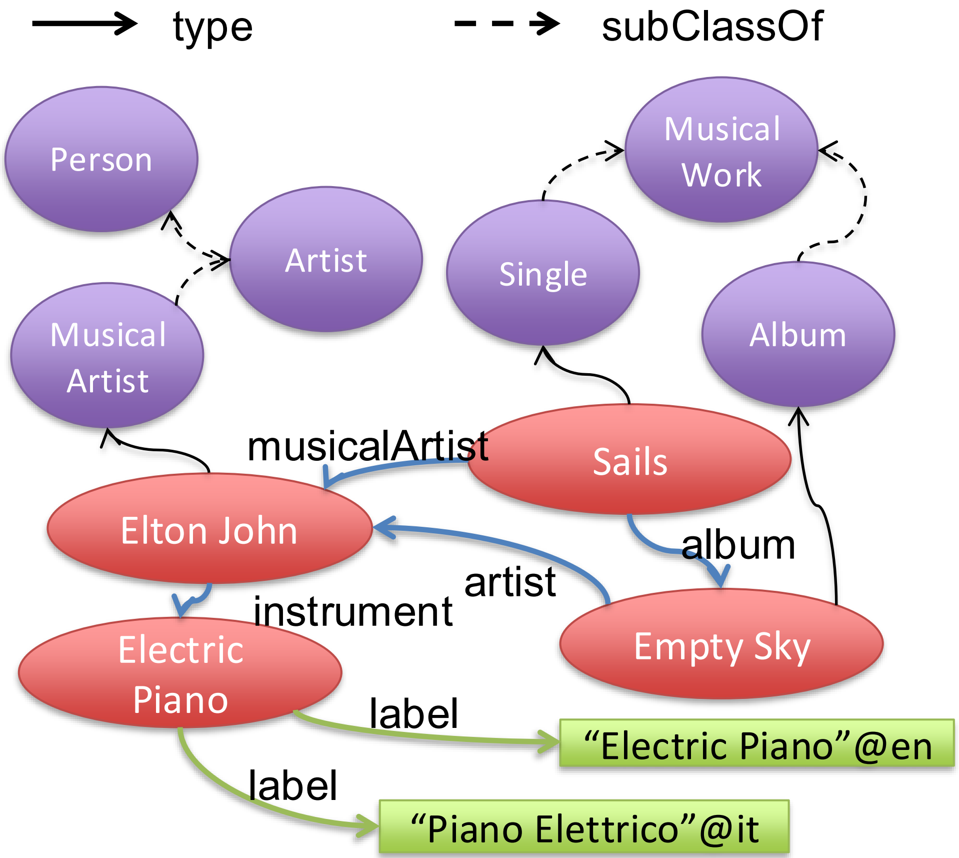
- RDF is a data model for representing data on the Web

  based on:

  - Triples – basic unit to organize the information
  - **Directed (labeled) graphs – sets of triples**
  - URIs – unique resource identifiers

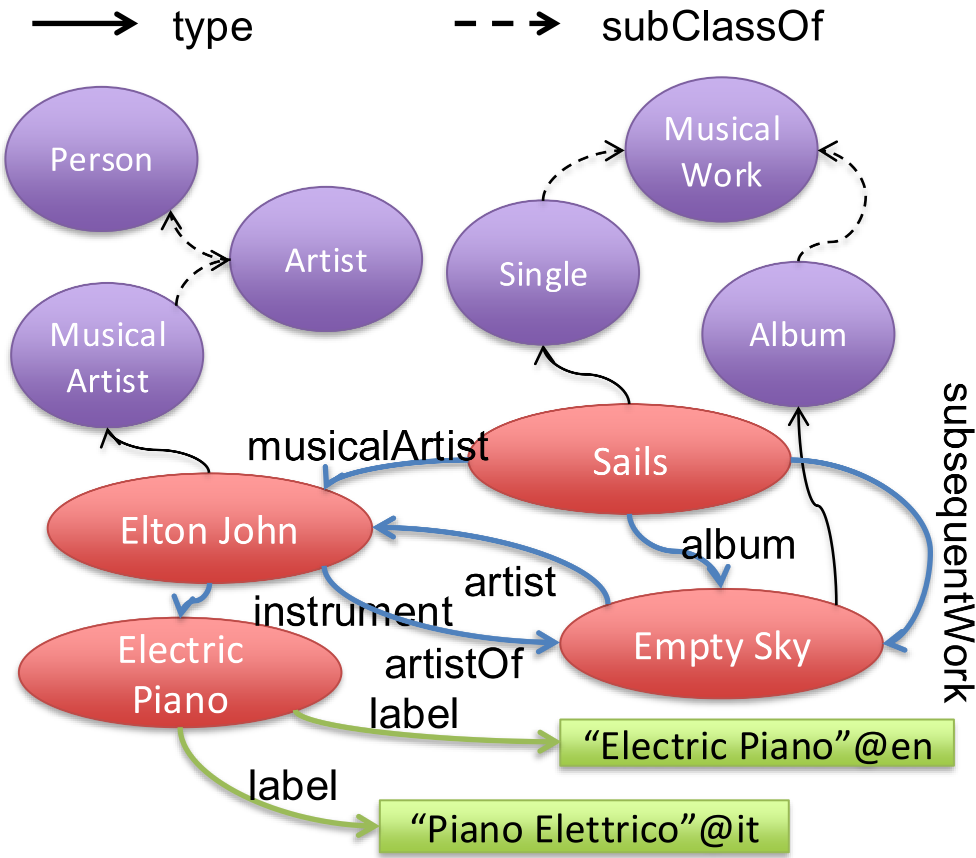
RDF: Graphs (vs. Triples)

<Electric Piano, label, “Electric Piano”@en>
<Electric Piano, label, “Piano Elettrico”@it>
<Elton John, instrument, Electric Piano>
<Empty Sky, artist, Elton John>
<Sails, musicalArtist, Elton John>
<Sails, album, Empty Sky>
<Elton John, instrument, Electric Piano>
<Elton John, type, Musical Artist>
<Sails, type, Single>
<Empty Sky, type, Album>
<Musical Artist, subClassOf, Artist>
<Artist, subClassOf, Person>
<Album, subClassOf, Musical Work>
<Single, subClassOf, Musical Work>



RDF: Graphs (vs. Triples)

<Electric Piano, label, "Electric Piano"@en>
<Electric Piano, label, "Piano Elettrico"@it>
<Elton John, instrument, Electric Piano>
<Empty Sky, artist, Elton John>
<Sails, musicalArtist, Elton John>
<Sails, album, Empty Sky>
<Elton John, instrument, Electric Piano>
<Elton John, type, Musical Artist>
<Sails, type, Single>
<Empty Sky, type, Album>
<Musical Artist, subClassOf, Artist>
<Artist, subClassOf, Person>
<Album, subClassOf, Musical Work>
<Single, subClassOf, Musical Work>
<Elton John, artistOf, Empty Sky>
<Sails, subsequentWork, Empty Sky>



RDF: Basic Principles

- RDF is a data model for representing data on the Web

  based on:

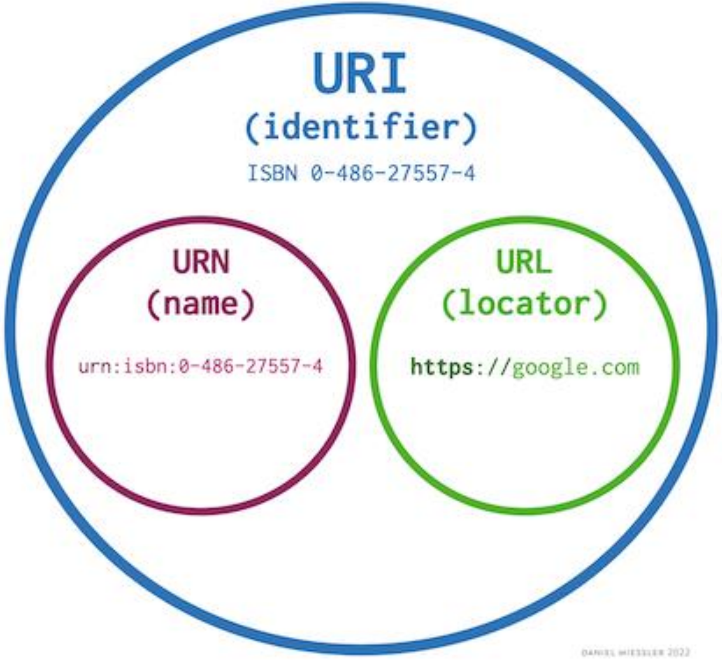
  - Triples – basic unit to organize the information
  - Directed (labeled) graphs – sets of triples
  - **URIs – unique resource identifiers**

Uniform Resource Identifiers (URI)

- **Uniform Resource Identifier (URI)**: a string of characters that uniquely identify a name or a resource on the internet. A URI identifies a resource by name, location, or both. URIs have two specializations known as Uniform Resource Locator (URL), and Uniform Resource Name (URN).

- **Uniform Resource Locator (URL)**: a type of URI that specifies not only a resource, but how to reach it on the internet—like `http://`, `ftp://`, or `mailto://`.

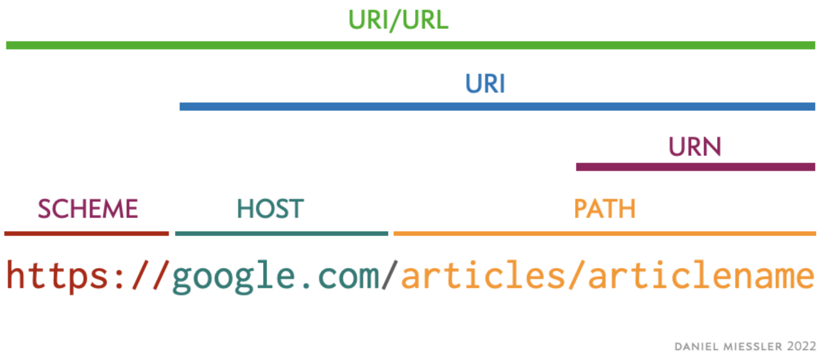
- **Uniform Resource Name (URN)**: a type of URI that uses the specific naming scheme of urn:—like `urn:isbn:0-486-27557-4` or `urn:isbn:0-395-36341-1`.

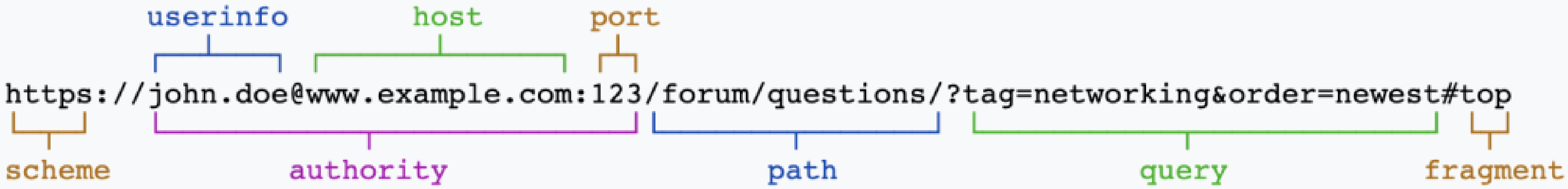


Source & more info: https://danielmiessler.com/study/difference-between-uri-url/

Uniform Resource Locators (URL)

- Scheme: the protocol that you're using to interact.
- Authority: the target you're accessing. This breaks down into userinfo, host, and port.
- Path: the resource you're requesting on the host.
- Query: the parameters being used within the web application. E.g.: https://google.com/search?s=bing
- Fragment: the target to jump to within a given page.

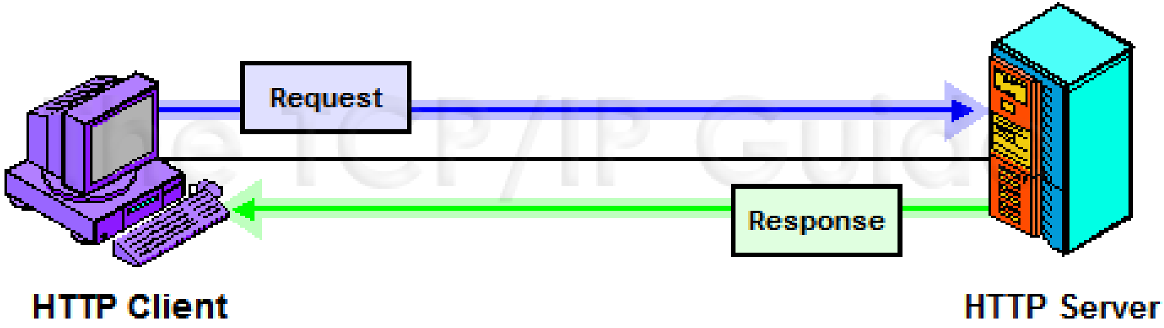




Source & more info: https://danielmiessler.com/study/difference-between-uri-url/

The Web = HTTP

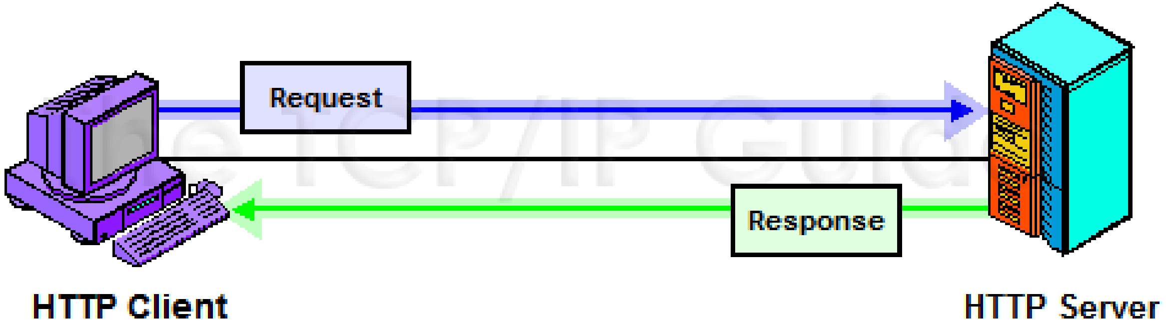
- Hypertext Transfer Protocol (HTTP) is an **application-layer protocol** for transmitting hypermedia documents, such as HTML. It was designed for communication between web browsers and web servers, but it can also be used for other purposes.
- HTTP follows a classical client-server model, with a client opening a connection to make a request, then waiting until it receives a response.
- HTTP is a stateless protocol, meaning that the server does not keep any data (state) between two requests.



Sources: https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP
http://www.tcpipguide.com/free/t_HTTPOperationalModelandClientServerCommunication.htm

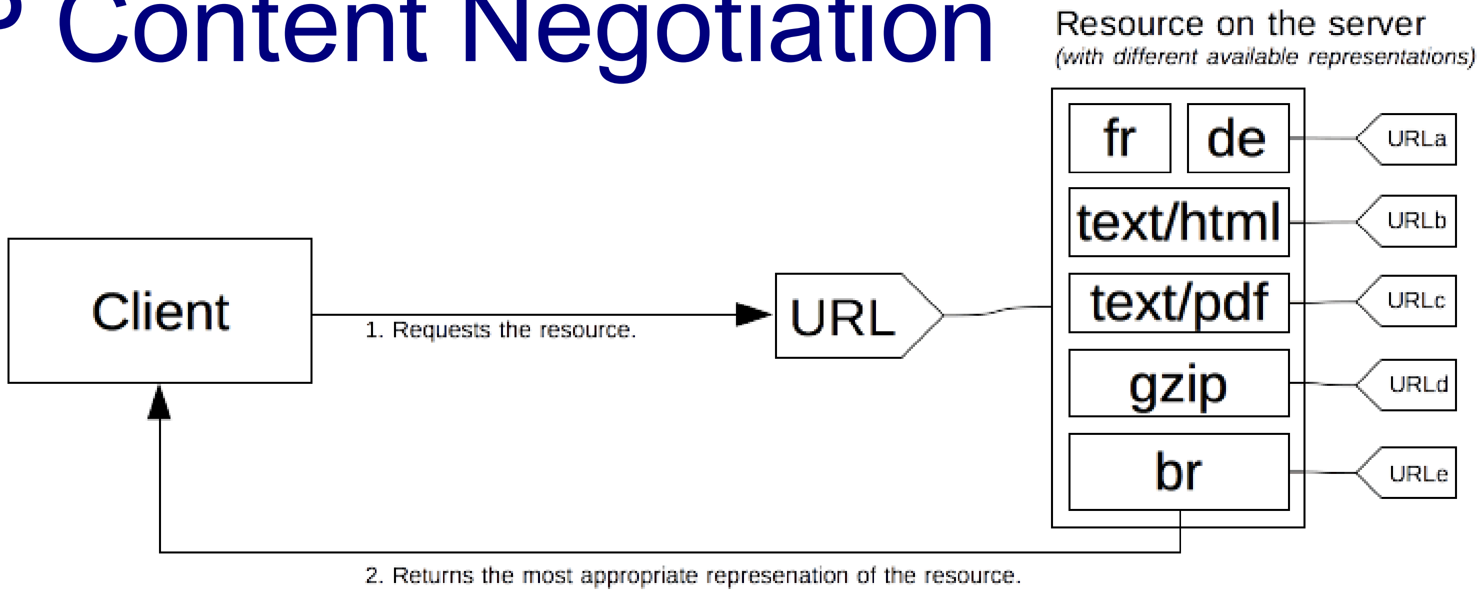
The Web = HTTP

- An extensible protocol that relies on concepts like resources and Uniform Resource Identifiers (URIs), simple message structure, and client-server communication flow.
- Numerous extensions have been developed over the years that add updated functionality and semantics with new HTTP methods or headers.
- More details also here: https://hackernoon.com/http-made-easy-understanding-the-web-client-server-communication-yz783vg3

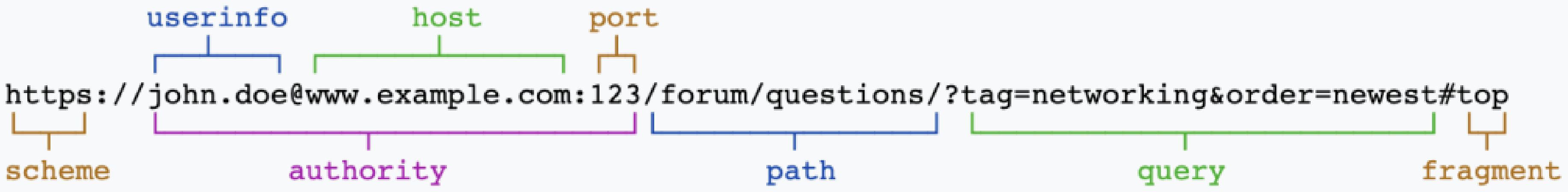


Sources: https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP
http://www.tcpipguide.com/free/t_HTTPOperationalModelandClientServerCommunication.htm

HTTP Content Negotiation



Source & more details: https://docs.w3cub.com/http/content_negotiation

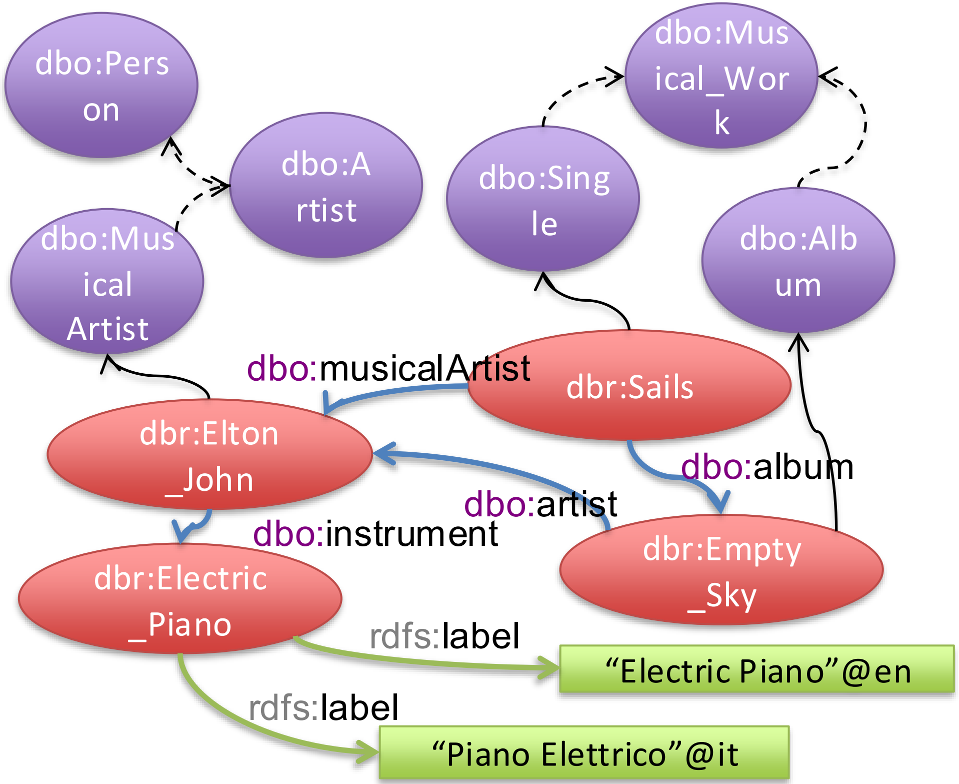


From URI to IRI

- URI: encoded with a subset of US-ASCII
- IRI: Internationalized Resource Identifiers
  - Superset of URIs
  - Universal Character Set (Unicode/ISO 10646)
    - allow content developers and users to identify resources in their own languages
  - The specification also discusses how to convert an IRI to a URI for resolution on existing systems
    - More info at: https://www.w3.org/2004/11/uri-iri-pressrelease

RDF: IRIfied Graphs and Triples

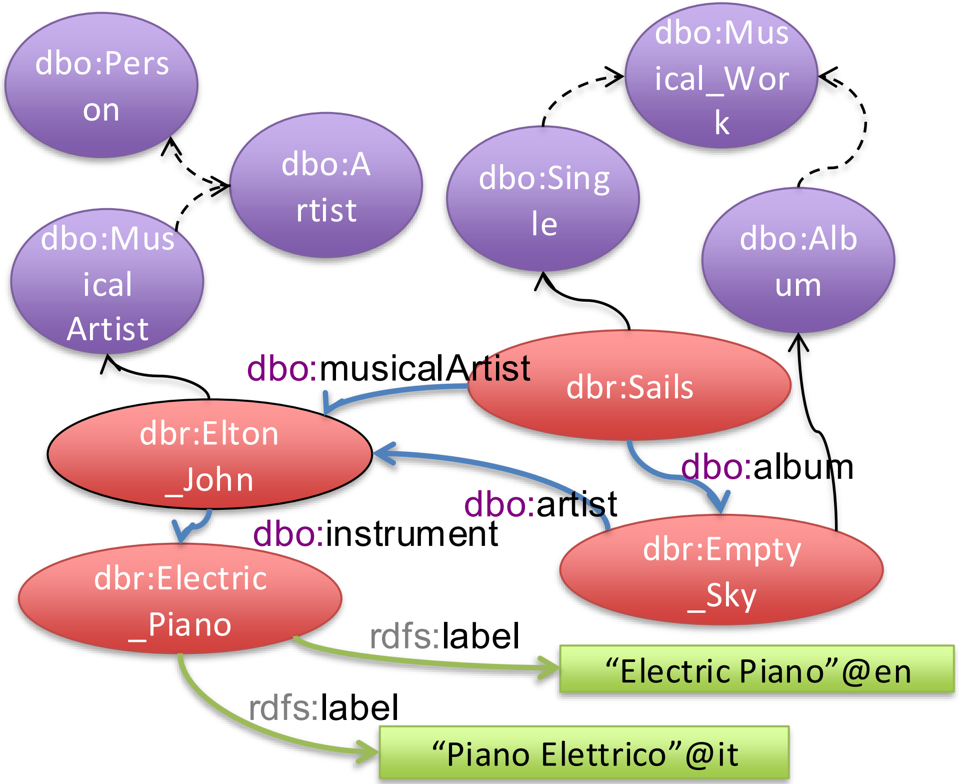
<dbr:Electric_Piano, rdfs:label, “Electric Piano”@en>
<dbr:Electric_Piano, rdfs:label, “Piano Elettrico”@it>
<dbr:Elton_John, dbo:instrument, dbr:Electric Piano>
<dbr:Empty_Sky, dbo:artist, dbr:Elton_John>
<dbr:Sails, dbo:musicalArtist, dbr:Elton_John>
<dbr:Sails, dbo:album, dbr:Empty_Sky>
<dbr:Elton_John, dbo:instrument, dbr:Electric_Piano>
<dbr:Elton_John, rdf:type, dbr:Musical_Artist>
<dbr:Sails, rdf:type, dbo:Single>
<dbr:Empty_Sky, rdf:type, dbo:Album>
<dbo:Musical_Artist, rdfs:subClassOf, dbo:Artist>
<dbo:Artist, rdfs:subClassOf, dbo:Person>
<dbo:Album, rdfs:subClassOf, dbo:Musical_Work>
<dbo:Single, rdfs:subClassOf, dbo:Musical_Work>



RDF: IRIfied Graphs and Triples

<dbr:Electric_Piano, rdfs:label, “Electric Piano”@en>
<dbr:Electric_Piano, rdfs:label, “Piano Elettrico”@it>
<dbr:Elton_John, dbo:instrument, dbr:Electric Piano>
<dbr:Empty_Sky, dbo:artist, dbr:Elton_John>
<dbr:Sails, dbo:musicalArtist, dbr:Elton_John>
<dbr:Sails, dbo:album, dbr:Empty_Sky>
<dbr:Elton_John, dbo:instrument, dbr:Electric_Piano>
<dbr:Elton_John, rdf:type, dbr:Musical_Artist>
<dbr:Sails, rdf:type, dbo:Single>
<dbr:Empty_Sky, rdf:type, dbo:Album>
<dbo:Musical_Artist, rdfs:subClassOf, dbo:Artist>
<dbo:Artist, rdfs:subClassOf, dbo:Person>
…

dbr: for http://dbpedia.org/resource/
e.g.: http://dbpedia.org/resource/Elton_John
dbo: for http://dbpedia.org/ontology/
rdfs for https://www.w3.org/2000/01/rdf-schema#
rdf for http://www.w3.org/1999/02/22-rdf-syntax-ns#



RDF Turtle/N-Triples

Check the full specification here: https://www.w3.org/TR/n-triples/

Basic syntax of RDF Turtle/N-Triples

- Three types of nodes (see next slide)
  - IRI, Blank nodes, Literal
- Constraints on where they can appear

Goal:
human & machine-readable

IRI

e.g. http://dbpedia.org/ontology/artist

e.g. dbo:artist

dot !

subject predicate object .

IRI or Blank node

e.g. <http://dbpedia.org/resource/Empty_Sky>

e.g. dbr:Empty_Sky

e.g. _:a

IRI, Blank node or Literal

e.g. <http://dbpedia.org/resource/Elton_John>

e.g. _:a

e.g. “Elton John”

RDF: IRIfied Graphs and Triples

Proper Turtle Syntax

In proper Turtle syntax: no prefixes

```
<http://dbpedia.org/resource/Electric_Piano> <https://www.w3.org/2000/01/rdf-schema#label> “Electric Piano”@en .
<http://dbpedia.org/resource/Electric_Piano> <https://www.w3.org/2000/01/rdf-schema#label> “Piano Elettrico”@it .
<http://dbpedia.org/resource/Elton_John> <http://dbpedia.org/ontology/instrument> <http://dbpedia.org/resource/Electric Piano> .
<http://dbpedia.org/resource/Empty_Sky> <http://dbpedia.org/ontology/artist> <http://dbpedia.org/resource/Elton_John> .
<http://dbpedia.org/resource/Sails> <http://dbpedia.org/ontology/musicalArtist> <http://dbpedia.org/resource/Elton_John> .
<http://dbpedia.org/resource/Sails> <http://dbpedia.org/ontology/album> <http://dbpedia.org/resource/Empty_Sky> .
<http://dbpedia.org/resource/Elton_John> <http://dbpedia.org/ontology/instrument> <http://dbpedia.org/resource/Electric_Piano> .
<http://dbpedia.org/resource/Elton_John> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://dbpedia.org/resource/Musical_Artist> .
<http://dbpedia.org/resource/Sails> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://dbpedia.org/ontology/Single> .
<http://dbpedia.org/resource/Empty_Sky> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://dbpedia.org/ontology/Album> .
<http://dbpedia.org/ontology/Musical_Artist> <https://www.w3.org/2000/01/rdf-schema#subClassOf> <http://dbpedia.org/ontology/Artist> .
<http://dbpedia.org/ontology/Artist> <https://www.w3.org/2000/01/rdf-schema#subClassOf> <http://dbpedia.org/ontology/Person> .
…
```

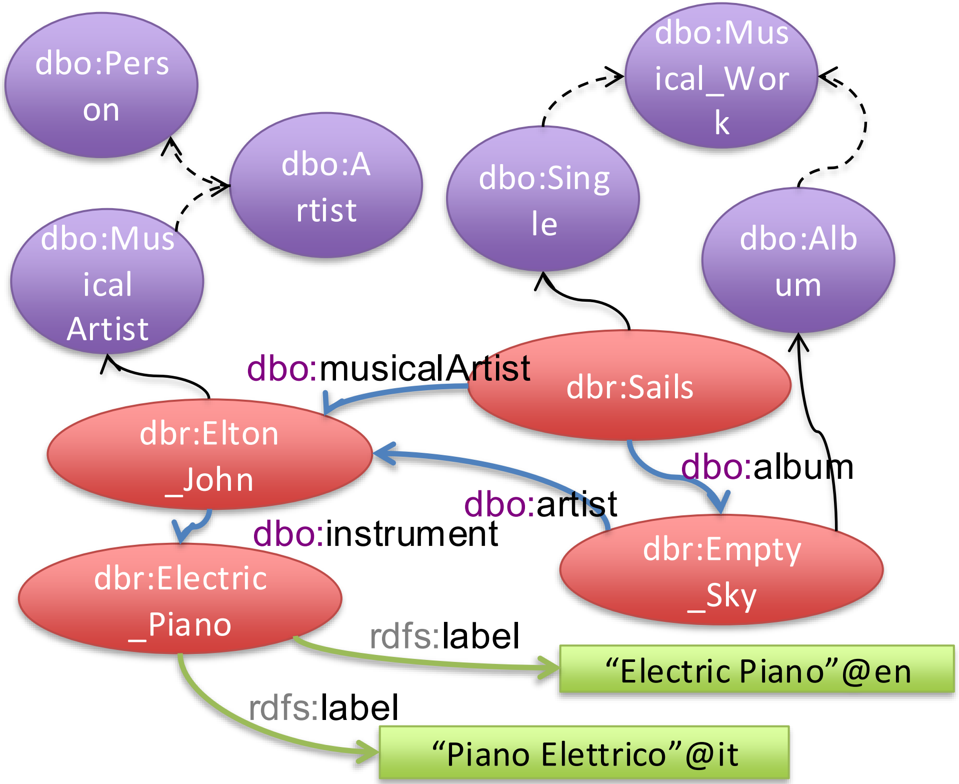
RDF: IRIfied Graphs and Triples

Proper Turtle Syntax

@prefix dbr: <http://dbpedia.org/resource/> .
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix rdfs: <https://www.w3.org/2000/01/rdf-schema#> .
@prefix rdf: http://www.w3.org/1999/02/22-rdf-syntax-ns# .

dbr:Electric_Piano rdfs:label "Electric Piano"@en .
dbr:Electric_Piano rdfs:label "Piano Elettrico"@it .
dbr:Elton_John dbo:instrument dbr:Electric Piano .
dbr:Empty_Sky dbo:artist dbr:Elton_John .
dbr:Sails dbo:musicalArtist dbr:Elton_John .
dbr:Sails dbo:album dbr:Empty_Sky .
dbr:Elton_John dbo:instrument dbr:Electric_Piano .
dbr:Elton_John rdf:type dbr:Musical_Artist .
dbr:Sails rdf:type dbo:Single .
dbr:Empty_Sky rdf:type dbo:Album .
dbo:Musical_Artist rdfs:subClassOf dbo:Artist .
dbo:Artist rdfs:subClassOf dbo:Person .
dbo:Album rdfs:subClassOf dbo:Musical_Work .
dbo:Single rdfs:subClassOf dbo:Musical_Work .

In proper Turtle syntax: with prefixes



RDF Turtle/N-Triples

IRIs (preferred) identifiers for individuals, properties, classes

<http://dbpedia.org/resource/Empty_Sky>

dc:title

dbo:artist

(Global) web-scale identifiers

Blank node

_:name

[]

(Local) dataset-level identifiers or *placeholders*

Literal

Strings, numbers, dates, …

- Plain literals

  "2006"

  "Electric Piano"

- Typed literals: literal + its datatype

  "2006"^^xsd:integer

  "Electric Piano"^^xsd:string

Datatypes

xsd:boolean (true, false)

xsd:integer (10, -1293)

xsd:decimal (12.13)

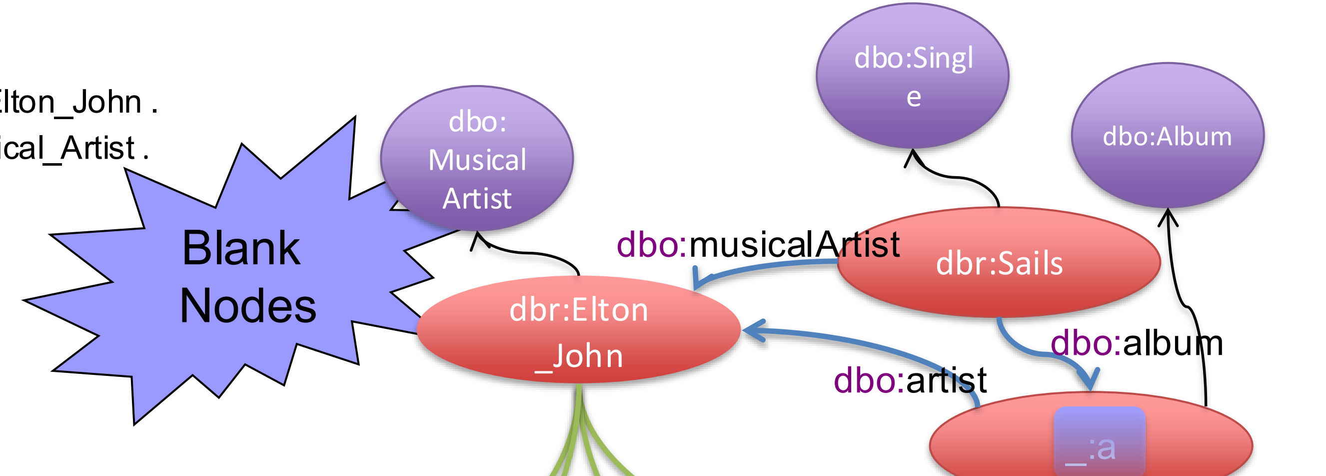
xsd:double (2.78e-2)

xsd:string ("Electric Piano")

Not identifiers, just "data", possibly with a datatype

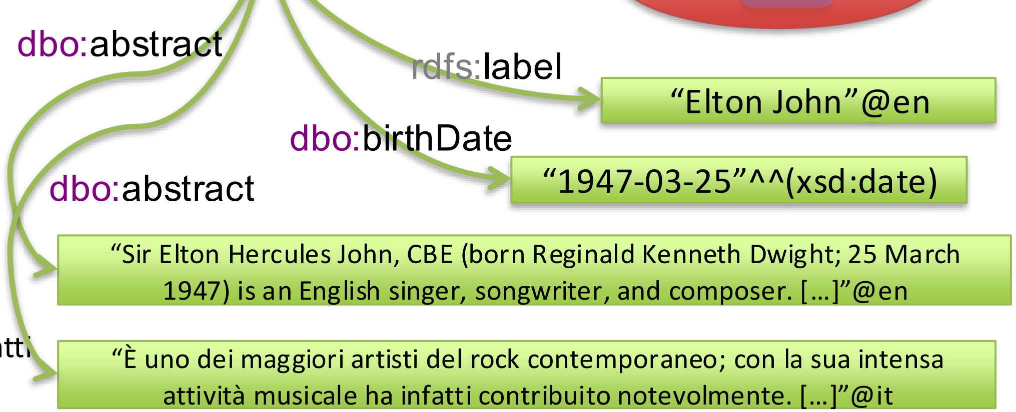
RDF: IRIfied Graphs + BN + Literals

dbr:Sails rdf:type dbo:Single .
dbr:Sails dbo:musicalArtist dbr:Elton_John .
dbr:Elton_John rdf:type dbr:Musical_Artist .
dbr:Sails dbo:album _:a .
_:a rdf:type dbo:Album .
_:a, dbo:artist, dbr:Elton_John .



dbr:Elton_John dbo:abstract
“Sir Elton Hercules John, CBE (born Reginald Kenneth Dwight; 25 March 1947) is an English singer, songwriter, and composer. […]”@en .

dbr:Elton_John dbo:abstract
““È uno dei maggiori artisti del rock contemporaneo; con la sua intensa attività musicale ha infatti contribuito notevolmente. […]”@it .



RDF Turtle/N-Triples

Check the full specification here:
https://www.w3.org/TR/n-triples/

Turtle also provides abbreviations:
https://www.w3.org/TR/turtle/
E.g., more triples with same subject and/or predicate

Goal:
human & machine-readable

IRI
e.g. http://dbpedia.org/ontology/artist
e.g. dbo:artist

dot !

subject predicate object .

IRI or Blank node
e.g. <http://dbpedia.org/resource/Empty_Sky>
e.g. dbr:Empty_Sky
e.g. _:a

IRI, Blank node or Literal
e.g. <http://dbpedia.org/resource/Elton_John>
e.g. _:a
e.g. "Elton John"

Turtle: Abbreviations

- **rdf:type**
  - dbr:Elton_John **a** dbr:Musical_Artist .
    - … equivalent to:
      - dbr:Elton_John rdf:type dbr:Musical_Artist .
- Same **subject**
  - dbr:Empty_Sky dbo:artist dbr:Elton_John **;** rdf:type dbo:Album .
    - … equivalent to:
      - dbr:Empty_Sky dbo:artist dbr:Elton_John .
      - dbr:Empty_Sky rdf:type dbo:Album .
- Same **subject and predicate**
  - dbr:Electric_Piano rdfs:label “Electric Piano”@en **,** “Piano Elettrico”@it .
    - … equivalent to:
      - dbr:Electric_Piano rdfs:label “Electric Piano”@en .
      - dbr:Electric_Piano rdfs:label “Piano Elettrico”@it .

RDF: N-Triples vs XML Syntax

The following N-Triples file:

```
<http://www.w3.org/2001/08/rdf-test/>  <http://purl.org/dc/elements/1.1/creator>    "Dave Beckett" .
<http://www.w3.org/2001/08/rdf-test/>  <http://purl.org/dc/elements/1.1/creator>    "Jan Grant" .
<http://www.w3.org/2001/08/rdf-test/>  <http://purl.org/dc/elements/1.1/publisher>  _:a .
_:a                                    <http://purl.org/dc/elements/1.1/title>      "World Wide Web Consortium" .
_:a                                    <http://purl.org/dc/elements/1.1/source>     <http://www.w3.org/> .
```

represents the same RDF graph as the following RDF/XML:

```
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:dc="http://purl.org/dc/elements/1.1/">
  <rdf:Description rdf:about="http://www.w3.org/2001/08/rdf-test/">
    <dc:creator>Jan Grant</dc:creator>
    <dc:creator>Dave Beckett</dc:creator>
    <dc:publisher>
      <rdf:Description>
        <dc:title>World Wide Web Consortium</dc:title>
        <dc:source rdf:resource="http://www.w3.org/"/>
      </rdf:Description>
    </dc:publisher>
  </rdf:Description>
</rdf:RDF>
```

RDF/XML Syntax: https://www.w3.org/TR/rdf-syntax-grammar/

RDF Graphs (KGBook)

- **RDF graph**
  - Set of RDF triples, manipulated as a labeled directed graph
  - Formal definition with query language
- **RDF data set**
  - Set of RDF triples. It is comprised of
    - One default graph
    - Zero or more named graphs

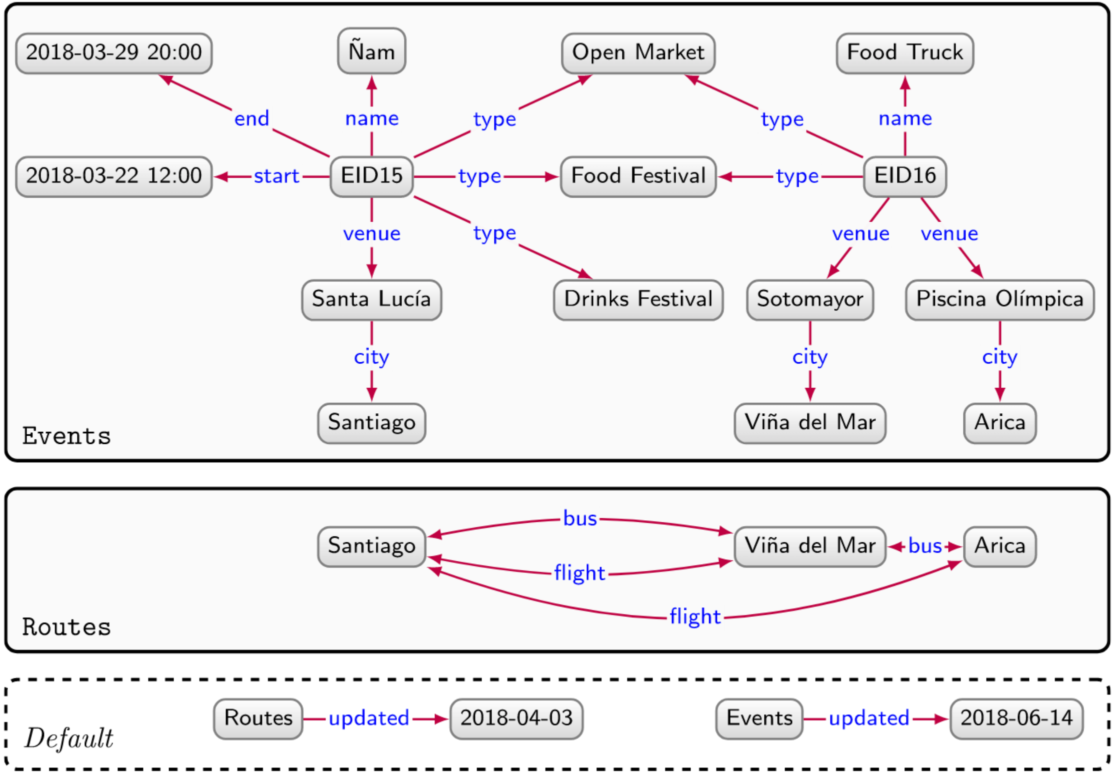


Figure 2.4: Graph dataset based on directed edge-labelled graphs with two named graphs and a default graph describing events and routes

RDF Graphs (KGBook)

Graph names

Rationale:

- instead of having one large graph, we can have many smaller graphs and exploit natural joins over RDF data - via IRIs;

How to name graphs:

- **Name of the graph**, i.e., the id specified in the beginning of the RDF file:
- **N-Quads format** (https://www.w3.org/TR/n-quads/) instead of triples
  - `<subject> <predicate> <object>` **`<context>`** `.`
  - `<subject> <predicate> <object>` **`<graphname>`** `.`
    - We use the <context> field to specify the graph name

RDF Characteristics

- Independence
  - Since predicates are resources, any independent organization can invent them.
- Interchange
  - Since RDF properties can be converted into XML, they are easy for us to interchange.
- Scalability
  - RDF properties are simple <subject, predicate, object> triple, so they are easy to handle and look things up by.
- Properties are resources
  - This means that they can have their own properties and can be found and manipulated like any other Resource.
- Subject and object can be resources
  - So, they can have properties too.

Homework: Match the Minimal KG Definition to RDF Language …

- A quintuple: <*E*,*L*,*T*,*P*,*A*>
  - *E*: a set of entity id symbols (= constants)
  - *L*: a set of literals, i.e., strings, numbers, etc. (= constants)
  - *T*: a set of type symbols (= unary predicate symbols)
  - *P*: a set of relation symbols (= binary predicate symbols)
  - *A*: a set of axioms in a language ….
- *A*: a set of **axioms** in a language (≈ subset of FOL)
  - A-facts:
    - Types of entities and literals: $\boldsymbol{t(e)}$ | $\boldsymbol{t(l)}$ - with $e \in E$ and $l \in L$
    - Relations between entities: $\boldsymbol{r(e_1, e_2)}$ | $\boldsymbol{r(e, l)}$ – with $e_i \in E$ and $l \in L$
  - A-general-axioms:
    - Depends on the language….
    - Minimal requirements: axioms with the form $\forall \boldsymbol{x} \; (\boldsymbol{t_1(x)} \rightarrow \boldsymbol{t_2(x)})$ that specify subtype dependencies that define a partial order over *T*